

scikit-learn Workflow

Практический конспект - Split - Preprocessing - Pipeline - Prediction - Leakage

Главная идея: Весь preprocessing и модель собираются в один Pipeline: fit учится только на train, predict безопасно работает с сырыми новыми данными.

Содержание

1. X и y	1
2. train_test_split	1
3. random_state	2
4. stratify	2
5. fit, transform и fit_transform	2
6. SimpleImputer	2
7. StandardScaler	2
8. LogisticRegression	3
9. Accuracy	3
10. Pipeline	3
11. OneHotEncoder	3
12. ColumnTransformer	3
13. handle_unknown='ignore'	4
14. Data leakage	4
15. Полный end-to-end workflow	4
16. Новый объект	4
17. Финальная шпаргалка	5
18. Что должно быть в голове	5

1. X и y

```
target = 'passed'
X = df.drop(columns=target)
y = df[target]

print(X.shape)
print(y.shape)
```

X содержит только признаки, y - целевую переменную. Target нельзя случайно оставить среди признаков.

Форма: X обычно имеет shape (n_samples, n_features), y - (n_samples,).

2. train_test_split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Train нужен для обучения всех параметров. Test имитирует новые данные и используется только для финальной проверки выбранного workflow.

3. random_state

Разбиение содержит случайность. Фиксированный **random_state** делает эксперимент воспроизводимым: одинаковые данные и код дают одинаковый split.

Важно: random_state не делает модель лучше и не должен подбираться ради высокой accuracy.

4. stratify

stratify=y сохраняет приблизительную долю классов в train и test. Это особенно важно при редком положительном классе.

```
print(y.value_counts(normalize=True))
print(y_train.value_counts(normalize=True))
print(y_test.value_counts(normalize=True))
```

Ограничение: При слишком редком классе stratify может быть невозможен: в каждом классе должно хватать объектов для обеих частей.

5. fit, transform и fit_transform

Метод	Смысл
fit(X_train)	Выучить параметры только по train
transform(X)	Применить уже выученное преобразование
fit_transform(X_train)	Выучить и сразу преобразовать train
<pre>prep.fit(X_train) X_train_ready = prep.transform(X_train) X_test_ready = prep.transform(X_test)</pre>	

Никогда: Не вызывай fit или fit_transform на test: это переносит информацию из будущей проверки в обучение.

6. SimpleImputer

```
from sklearn.impute import SimpleImputer

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

X_train_num = num_imputer.fit_transform(X_train[num_cols])
X_test_num = num_imputer.transform(X_test[num_cols])
```

Стратегии: **mean, median, most_frequent, constant**. Медиана устойчивее к выбросам.

7. StandardScaler

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_num)
X_test_scaled = scaler.transform(X_test_num)
```

Для каждого числового feature scaler учит mean и standard deviation на train, затем применяет $z = (x - \text{mean}) / \text{std}$.

Когда полезно: Масштабирование важно для Logistic Regression, KNN, SVM и многих методов оптимизации; деревьям обычно не требуется.

8. LogisticRegression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
y_proba = model.predict_proba(X_test_scaled)[: , 1]
```

predict возвращает класс. **predict_proba** возвращает вероятности классов; колонка [, 1] обычно соответствует положительному классу, но порядок можно проверить в **model.classes_**.

9. Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print(f'accuracy = {accuracy:.3f}')
```

Accuracy - доля правильных ответов. При дисбалансе она может вводить в заблуждение: предсказание только частого класса уже даст высокий результат.

Смысл: Метрика оценивается на данных, которые не участвовали в fit.

10. Pipeline

```
from sklearn.pipeline import Pipeline

num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
```

Pipeline хранит порядок шагов и гарантирует, что при **fit** они учатся, а при **predict** только применяются. Это снижает риск расхождения между train и production.

11. OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder

cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore')),
])
```

One-hot создаёт отдельный бинарный столбец для каждой категории. Категории обучаются только на train.

12. ColumnTransformer

```
from sklearn.compose import ColumnTransformer

num_cols = ['age', 'study_hours', 'score']
cat_cols = ['city']

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols),
])
```

Модель: Числовые и категориальные столбцы проходят разные ветки, затем результаты соединяются в одну матрицу признаков.

13. handle_unknown='ignore'

В test или production может появиться город, которого не было в train. С **handle_unknown='ignore'** encoder не падает: для неизвестной категории все обученные one-hot столбцы этой переменной будут равны нулю.

```
new_student = pd.DataFrame([{'age': 20, 'study_hours': 12, 'score': 78, 'city': 'London'}])
```

Важно: ignore предотвращает ошибку, но не сообщает модели смысл новой категории. Частые unknown значения нужно мониторить.

14. Data leakage

Утечка	Правильный вариант
imputer.fit на всём X	fit внутри Pipeline после split
scaler.fit до split	fit только на X_train
ONE обучен на test	categories только из train
Feature содержит будущий ответ	Удалить или пересчитать на момент прогноза

Граница: Любой шаг, который учит статистики или категории, должен находиться внутри train-only workflow.

15. Полный end-to-end workflow



```
final_model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LogisticRegression(max_iter=1000)),
])

final_model.fit(X_train, y_train)
y_pred = final_model.predict(X_test)
y_proba = final_model.predict_proba(X_test)[:, 1]
accuracy = accuracy_score(y_test, y_pred)
```

Результат: fit обучает preprocessing + model. predict сам применяет те же преобразования к сырым новым данным.

16. Новый объект

```
pred = final_model.predict(new_student)[0]
proba = final_model.predict_proba(new_student)[0, 1]

print('class:', pred)
print('P(passed=1):', round(proba, 3))
```

Новый DataFrame должен иметь те же исходные названия столбцов и совместимые типы. Вручную масштабировать или one-hot кодировать его не нужно.

17. Финальная шпаргалка

```
X = df.drop(columns='target'); y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, random_state=42, stratify=y
)

num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore')),
])
prep = ColumnTransformer([('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols)])
final_model = Pipeline([('prep', prep), ('model', LogisticRegression(max_iter=1000))])
final_model.fit(X_train, y_train)
accuracy_score(y_test, final_model.predict(X_test))
```

18. Что должно быть в голове

Workflow: raw DataFrame -> X/y -> split -> preprocessing -> model -> prediction -> metric

fit/transform: fit учит параметры; transform применяет; test получает только transform/predict.

Защита: Pipeline + ColumnTransformer объединяют все train-only преобразования и уменьшают риск leakage.